

CO2-AT2-ASSIGNMENT 1

← Back

QUESTIONS

Q1
University Alumni Information Management System
50 marks

1/1 answered

Q1 University Alumni Information Management System

50 marks

A prestigious university maintains an alumni database containing graduates working across different industries worldwide. Alumni details are periodically updated through departmental coordinators, alumni associations, and online portals. As a result, duplicate alumni IDs, repeated email addresses, inconsistent company names, and multiple contact records are present. The university intends to develop a Java application that manages alumni records efficiently and generates analytical reports using Collection Framework classes.

Develop a Java application that performs the following operations:

- Read Alumni ID, Name, Department, Graduation Year, Company Name, Designation, Email ID, and Country.
- Use String Handling methods to standardize names and company names.
- Store alumni records using an ArrayList.
- Remove duplicate Alumni IDs using a HashSet.
- Store Alumni IDs and Alumni Objects using a HashMap.
- Traverse records using Iterator.
- Use Generics in all collection implementations.
- Generate department-wise alumni reports and compare the functionality of List, Set, and Map.

Your Code

```
1 import java.util.*;
2
3 // Alumni class to hold graduate details
4 class Alumni {
5     private String id;
6     private String name;
7     private String department;
8     private int graduationYear;
9     private String company;
10    private String designation;
11    private String email;
12    private String country;
13
14    public Alumni(String id, String name, String department, int graduationYear, String company, String designation, String email, String country) {
15        this.id = id;
16        this.name = name;
17        this.department = department;
18        this.graduationYear = graduationYear;
19        this.company = company;
20        this.designation = designation;
21        this.email = email;
22        this.country = country;
23    }
24
25    // (b) String handling methods to standardize Name and Company
26    public String getId() { return id; }
27    public String getName() { return name; }
28    public String getDepartment() { return department; }
29    public int getGraduationYear() { return graduationYear; }
30    public String getCompany() { return company; }
```

Input

stdin input

Output

```

23 public String getCompany() { return company; }
24 public String getDesignation() { return designation; }
25 public String getEmail() { return email; }
26 public String getCountry() { return country; }
27
28 @Override
29 public String toString() {
30     return "ID: " + id + " | Name: " + name + " | Dept: " + department
31         + " | Year: " + graduationYear + " | Company: " + company
32         + " | Role: " + designation + " | Email: " + email + " |
33 }
34 }
35
36 public class Main {
37     public static void main(String[] args) {
38         // (g) Using Generics in all collection implementations
39
40         // (c) Store initial alumni records using ArrayList
41         List<Alumni> rawAlumniList = new ArrayList<>();
42
43         // (a) Sample inputs with duplicate IDs and inconsistent strings
44         rawAlumniList.add(new Alumni("ALU001", "alice smith", "Computer Science", 2018, "ABC Corp", "Software Engineer", "alice.smith@abc.com", "USA"));
45         rawAlumniList.add(new Alumni("ALU002", "bob johnson", "Electrical Engineering", 2019, "XYZ Inc", "Electrical Engineer", "bob.johnson@xyz.com", "Canada"));
46         rawAlumniList.add(new Alumni("ALU001", "Alice S.", "Computer Science", 2018, "ABC Corp", "Software Engineer", "alice.smith@abc.com", "USA"));
47         rawAlumniList.add(new Alumni("ALU003", "charlie brown", "Computer Science", 2020, "DEF Ltd", "Data Analyst", "charlie.brown@def.com", "UK"));
48         rawAlumniList.add(new Alumni("ALU004", "david lee", "Mechanical Engineering", 2017, "GHI Co", "Mechanical Engineer", "david.lee@ghi.com", "Australia"));
49
50         // (d) Remove duplicate Alumni IDs using HashSet
51         Set<String> uniqueIds = new HashSet<>();
52         List<Alumni> uniqueAlumniList = new ArrayList<>();
53
54         Iterator<Alumni> listIterator = rawAlumniList.iterator(); // (f)
55         while (listIterator.hasNext()) {
56             Alumni alumni = listIterator.next();
57             if (!uniqueIds.contains(alumni.getId())) {
58                 uniqueIds.add(alumni.getId());
59                 uniqueAlumniList.add(alumni);
60             }
61         }
62
63         // (e) Store Alumni IDs and Alumni Objects using HashMap
64         Map<String, Alumni> alumniMap = new HashMap<>();
65         Iterator<Alumni> uniqueIterator = uniqueAlumniList.iterator();
66         while (uniqueIterator.hasNext()) {
67             Alumni alumni = uniqueIterator.next();
68             alumniMap.put(alumni.getId(), alumni);
69         }
70
71         System.out.println("=== Standardised Unique Alumni Records (Map) ===");
72         Iterator<Map.Entry<String, Alumni>> mapIterator = alumniMap.entrySet().iterator();
73         while (mapIterator.hasNext()) {
74             Map.Entry<String, Alumni> entry = mapIterator.next();
75             System.out.println("Key [" + entry.getKey() + "] -> " + entry.getValue());
76         }
77
78         // (h) Generate department-wise alumni reports
79         System.out.println("\n=== Department-Wise Alumni Report ===");
80         Map<String, List<Alumni>> departmentMap = new HashMap<>();
81
82         for (Alumni alumni : alumniMap.values()) {
83             departmentMap.putIfAbsent(alumni.getDepartment(), new ArrayList<>());
84             departmentMap.get(alumni.getDepartment()).add(alumni);
85         }
86     }
87 }

```

```

73 // (e) Store Alumni IDs and Alumni Objects using HashMap
74 Map<String, Alumni> alumniMap = new HashMap<>();
75 Iterator<Alumni> uniqueIterator = uniqueAlumniList.iterator();
76 while (uniqueIterator.hasNext()) {
77     Alumni alumni = uniqueIterator.next();
78     alumniMap.put(alumni.getId(), alumni);
79 }
80
81 System.out.println("==== Standardised Unique Alumni Records (Map)");
82 Iterator<Map.Entry<String, Alumni>> mapIterator = alumniMap.entrySet().iterator();
83 while (mapIterator.hasNext()) {
84     Map.Entry<String, Alumni> entry = mapIterator.next();
85     System.out.println("Key [" + entry.getKey() + "] -> " + entry.getValue());
86 }
87
88 // (h) Generate department-wise alumni reports
89 System.out.println("\n==== Department-Wise Alumni Report ====");
90 Map<String, List<Alumni>> departmentMap = new HashMap<>();
91
92 for (Alumni alumni : alumniMap.values()) {
93     departmentMap.putIfAbsent(alumni.getDepartment(), new ArrayList<>());
94     departmentMap.get(alumni.getDepartment()).add(alumni);
95 }
96
97 Iterator<Map.Entry<String, List<Alumni>>> deptIterator = departmentMap.entrySet().iterator();
98 while (deptIterator.hasNext()) {
99     Map.Entry<String, List<Alumni>> deptEntry = deptIterator.next();
100     System.out.println("\nDepartment: " + deptEntry.getKey());
101     Iterator<Alumni> deptAlumniIterator = deptEntry.getValue().iterator();
102     while (deptAlumniIterator.hasNext()) {
103         System.out.println("    - " + deptAlumniIterator.next().getId());
104     }
105 }
106
107 // (h) Print Comparison of List, Set, and Map
108 printComparison();
109 }
110
111 // Function to compare functionality of List, Set, and Map
112 private static void printComparison() {
113     System.out.println("\n===== COMPARISON OF LIST, SET, AND MAP =====");
114     System.out.println("Feature | List | Set | Map");
115     System.out.println("-----|-----|-----|-----");
116     System.out.println("Ordering | Maintains insertion order | Does not maintain order | Maintains insertion order");
117     System.out.println("Duplicates | Allows duplicates | Does not allow duplicates | Allows duplicates");
118     System.out.println("Access | Index-based access | Index-based access | Index-based access");
119     System.out.println("Use Case | Maintaining sequential data | Maintaining unique elements | Maintaining key-value pairs");
120     System.out.println("Input data | IDs | IDs | IDs");
121 }
122 }
123 }
124 }
125 }

```

Run

Save

Submit